Semaine 0 – Choix techniques et architecture du laboratoire

4 mai 2026

Table des matières

1 Introduction **2**

2 Topologie générale du laboratoire virtualisé **2**
2.1 Composition des machines virtuelles 2
2.2 Justification du réseau interne 3

3 Écarts par rapport au plan initial du projet **3**
3.1 Remplacement d'Alpine Linux par OPNsense pour le pare-feu 3
3.2 Regroupement du moteur Python et de Wazuh sur une seule VM 3
3.3 Utilisation de machines virtuelles complètes plutôt que de conteneurs pour la cible 4

4 Flux de logs et configuration initiale **4**
4.1 Depuis le pare-feu OPNsense 4
4.2 Depuis la machine cible Debian 4
4.3 Réception sur le serveur SOC 4

5 Rôle de l'hôte Fedora dans le développement **5**

6 Gestion de la stack Wazuh (phases 8–10) **6**
6.1 Choix de Docker 6
6.2 Configuration Docker et limites mémoire 6
6.3 Ingestion des logs par Wazuh 7

7 Bilan de l'empreinte mémoire prévisionnelle **7**

8 Automatisation du laboratoire (Infrastructure as Code) **8**

9 Recommandations techniques complémentaires **8**
9.1 Snapshots Libvirt 8
9.2 Synchronisation temporelle 8
9.3 Journalisation des décisions 8

10 Conclusion de la semaine 0 **8**

1 Introduction

Ce document constitue le livrable de la **semaine 0** du projet . Il détaille les décisions techniques prises pour l'architecture du laboratoire virtualisé, les choix des systèmes d'exploitation, l'allocation des ressources mémoire, la topologie réseau et la gestion des flux de logs. Ces choix sont guidés par les contraintes du projet : budget matériel limité (16 Go de RAM sur l'hôte Fedora-Workstation(*version 43*)), nécessité de simuler un environnement de PME togolaise, et obligation de comparer ultérieurement le moteur Python développé avec la solution Wazuh dans des conditions contrôlées.

Toutes les décisions documentées ici s'appuient sur le document de projet initial (`ProjetFinal.pdf`) mais en adaptant certains aspects pour améliorer le réalisme et la robustesse du laboratoire.

2 Topologie générale du laboratoire virtualisé

Le laboratoire est construit sous **Libvirt 11.6.0** sur un hôte physique équipé de 16 Go de RAM et du système **Fedora-Workstation**. Les machines virtuelles (VMs) communiquent via un réseau interne VirtualBox (`vboxnet0`) en mode *Host-Only*, complètement isolé de l'Internet direct (sauf pour les mises à jour initiales, via une interface NAT temporaire).

2.1 Composition des machines virtuelles

Table 1 – Composition du laboratoire – VMs et allocation mémoire

Composant	**OS**	**RAM**	**IP**	**Rôle principal**
Firewall / Routeur	OPNsense 24.7	1 Go	10.0.1.1 (LAN)	Routage, segmentation réseau, génération de logs de pare-feu
Serveur SOC	Ubuntu Server 24.04	6 Go	10.0.1.10	— Collecte centralisée des logs — Moteur de corrélation Python (phases 1–7) — Stack Wazuh (Docker, phases 8–10)
Serveur Cible	Debian 12	1 Go	10.0.1.20	— Service SSH (cible brute-force) — Service de fichiers (FTP/SMB pour scénario d'exfiltration)
Machine attaquante	Kali Linux	2 Go	10.0.1.50	Simulation des scénarios d'attaque (brute-force SSH, scan réseau, exfiltration)

→ **Décision :** **Ajustement de la RAM du SOC :** La proposition initiale prévoyait 8 Go pour cette VM. Afin de maintenir une marge de sécurité confortable pour l'hôte

Xubuntu (qui consomme ≈ 3 Go avec VS Code et Wireshark), la RAM allouée a été réduite à **6 Go**. Cela reste largement suffisant : le moteur Python n'utilise que 1,5–2 Go, et Wazuh sera limité à 4 Go maximum via les contraintes Docker (voir section 6).

2.2 Justification du réseau interne

Le sous-réseau `10.0.1.0/24` est utilisé en mode *Host-Only* avec l'option *Internal Network* de VirtualBox. Cela garantit :

- Une isolation totale du laboratoire vis-à-vis du réseau domestique ou universitaire.
- La possibilité de capturer le trafic sur l'interface `vboxnet0` avec Wireshark depuis l'hôte, pour déboguer les flux de logs.
- Une configuration statique reproductible (pas de DHCP parasite).

3 Écarts par rapport au plan initial du projet

Le document de référence (`mini-soc-rapport.pdf`) proposait une architecture avec une VM Alpine Linux (512 Mo) et des conteneurs Docker pour la collecte et la cible. Les écarts suivants ont été délibérément introduits pour améliorer le réalisme et la robustesse du laboratoire.

3.1 Remplacement d'Alpine Linux par OPNsense pour le pare-feu

Motivation :

- OPNsense offre une interface web complète pour configurer les règles de pare-feu, le NAT et la redirection de logs (syslog). Cela accélère la mise en place et correspond davantage aux appliances utilisées dans les PME (même si souvent en matériel dédié).
- La surcharge mémoire (1 Go contre 512 Mo) est acceptable dans le budget global et ne compromet pas les objectifs de performance.

3.2 Regroupement du moteur Python et de Wazuh sur une seule VM

Motivation :

- Simplification de la gestion des logs : toutes les sources (firewall, cible) envoient leurs flux vers une unique IP (`10.0.1.10`), que ce soit pour le moteur Python ou pour Wazuh.
- **Isolation temporelle stricte** : durant les semaines 1 à 7, la stack Wazuh reste arrêtée (`docker-compose stop`) ; durant les semaines 8 à 10, le moteur Python est stoppé et seul Wazuh est actif. Les deux systèmes ne s'exécutent **jamais simultanément**.
- Les mesures de performance (RAM, CPU, latence) seront donc comparables à celles qu'on obtiendrait avec deux machines distinctes.

3.3 Utilisation de machines virtuelles complètes plutôt que de conteneurs pour la cible

Motivation :

- Une PME togolaise typique utilise un serveur physique ou un VPS (Virtual Private Server) exécutant un OS complet (Linux). Une VM Debian reproduit plus fidèlement ce contexte qu'un conteneur Docker léger.
- Les logs générés (`/var/log/auth.log`) sont strictement identiques à ceux d'un système réel.

4 Flux de logs et configuration initiale

Pour alimenter le moteur de corrélation (puis Wazuh), deux sources principales de logs sont configurées dès la semaine 0.

4.1 Depuis le pare-feu OPNsense

- **Configuration :** `System` → `Settings` → `Logging / Targets`
- **Cible :** `10.0.1.10` port `514` (UDP)
- **Niveau de log :** `Notice` et supérieur
- **Contenu :** événements de pare-feu (blocages, scans détectés via les règles IDS/IPS optionnelles)

→ Décision : Bien qu'OPNsense puisse générer un volume important de logs, il est conseillé de conserver le niveau `Notice` pour ne pas perdre d'événements de sécurité pertinents. Le moteur Python filtrera les champs nécessaires lors de la normalisation.

4.2 Depuis la machine cible Debian

- **Fichier de configuration :** `/etc/rsyslog.d/50-forward.conf`
- **Contenu :**

```
# Transfère uniquement les logs d'authentification
auth,authpriv.*  @10.0.1.10:514
```

- **Redémarrage :** `systemctl restart rsyslog`

→ Décision : **Pourquoi ne transférer que `auth,authpriv` ?** Cela évite de saturer le collecteur avec des logs inutiles (cron, debug, messages noyau) et garantit que le moteur Python reçoit précisément les événements nécessaires au scénario de brute-force SSH. Si d'autres scénarios l'exigent plus tard, on pourra ajouter d'autres *facilities*.

4.3 Réception sur le serveur SOC

Sur la VM SOC, le service `rsyslog` est configuré pour écouter sur le port UDP 514 et écrire les logs reçus dans `/var/log/remote/`. Le collecteur Python (développé à partir de la semaine 3) pourra soit lire directement ces fichiers, soit être alimenté via un socket Unix.

5 Rôle de l'hôte Fedora dans le développement

L'hôte physique **n'exécute aucun service de sécurité**. Il est exclusivement dédié aux tâches suivantes :

— **Édition de code :** Visual Studio Code avec l'extension `Remote - SSH`. Cela permet de travailler sur le code source localement tout en l'exécutant et le testant directement sur la VM SOC.

— **Gestion de versions :** Le dépôt Git `mini-soc-togo` est cloné sur l'hôte et synchronisé régulièrement avec un dépôt distant privé (GitLab).

— **Analyse réseau :** Wireshark est utilisé pour capturer le trafic sur l'interface virtuelle `vboxnet0` et vérifier que les paquets syslog arrivent correctement sur le port 514.

— **Accès console :** Toutes les VMs sont administrées en SSH depuis le terminal de l'hôte. Les interfaces graphiques de VirtualBox restent fermées (mode *headless*) pour économiser la RAM.

— **Rédaction du rapport :** Le document LaTeX du projet est maintenu sur l'hôte.

Cette séparation nette entre environnement de développement et environnement d'exécution assure que les mesures de performance (RAM, CPU, latence) ne seront pas faussées par les outils bureautiques de l'hôte.

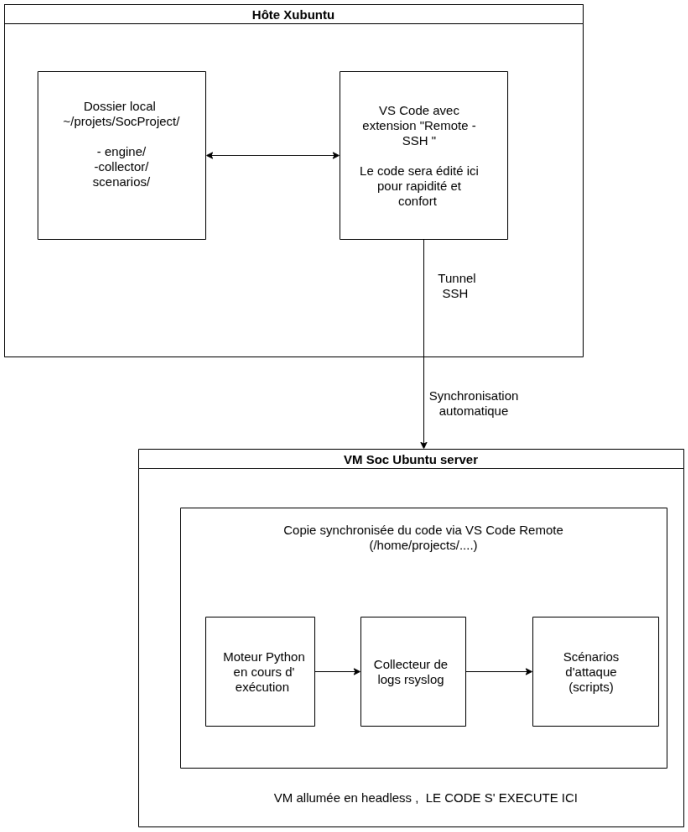

FIGURE 1 – Workflow de développement : édition locale sur l'hôte Xubuntu, exécution distante sur la VM SOC via SSH.

6 Gestion de la stack Wazuh (phases 8–10)

Pour le benchmark comparatif, Wazuh sera déployé sur la **VM SOC** à l'aide de `docker-compose` (version officielle fournie par Wazuh).

6.1 Choix de Docker

L'installation manuelle (*bare metal*) de Wazuh (Manager + Indexer + Dashboard) est complexe et consommatrice de ressources. L'approche conteneurisée offre plusieurs avantages :

— **Reproductibilité :** un simple `docker-compose up -d` lance toute la stack.

— **Limitation précise des ressources :** on peut imposer des quotas mémoire et CPU par conteneur, ce qui est crucial pour rester dans l'enveloppe des 6 Go de la VM.

— **Isolation :** les dépendances (Java pour OpenSearch, Python pour le manager) restent confinées.

— **Mode dormant :** `docker-compose stop` arrête tous les processus et libère intégralement la RAM. La commande sera exécutée à la fin de chaque session de test pendant les semaines 8–10.

6.2 Configuration Docker et limites mémoire

Extrait du fichier `docker-compose.yml` prévu :

```
services:
  wazuh-indexer:
    image: wazuh/wazuh-indexer:4.7.3
    deploy:
      resources:
        limits:
          memory: 3G
    volumes:
      - ./wazuh-data/indexer:/usr/share/wazuh-indexer/data

  wazuh-manager:
    image: wazuh/wazuh-manager:4.7.3
    deploy:
      resources:
        limits:
          memory: 1G
    ports:
      - "514:514/udp"   # Réception syslog
    volumes:
      - ./wazuh-data/manager:/var/ossec/data

  wazuh-dashboard:
    image: wazuh/wazuh-dashboard:4.7.3
    deploy:
      resources:
```

```
      limits:
        memory: 512M
  ports:
    - "5601:5601"
```

→ **Décision :** **Validation des limites mémoire :** Les valeurs ci-dessus (3 Go pour l'indexer, 1 Go pour le manager) sont des **maximums absolus**. En pratique, la consommation réelle pourra être surveillée avec `docker stats` et éventuellement ajustée à la baisse si la machine montre des signes de *swap*. L'important est que la somme des limites n'excède pas la RAM disponible sur la VM (6 Go).

6.3 Ingestion des logs par Wazuh

Le manager Wazuh écoutera sur le port UDP 514 (mappé sur l'hôte Docker) grâce au module `remote`. Une règle de décodage personnalisée sera écrite pour analyser les logs syslog provenant d'OPNsense et de la cible Debian, reproduisant ainsi le même flux que celui traité par le moteur Python.

7 Bilan de l'empreinte mémoire prévisionnelle

Le tableau 2 détaille la consommation mémoire prévue pour les deux grandes phases du projet.

Table 2 – Budget RAM du laboratoire par phase

Composant	**Phases 1–7 (dév. moteur)**	**Phases 8–10 (benchr**
Firewall OPNsense	1 Go	1 Go
Serveur SOC (OS + services de base)	1 Go	1 Go
Moteur Python (en charge)	1,5 Go	0 Go (arrêté)
Wazuh (Indexer + Manager + Dashboard)	0 Go	$\leq 4,5$ Go (limites Do
Serveur Cible Debian	1 Go	1 Go
Machine attaquante Kali	2 Go	2 Go
Total VMs	**6,5 Go**	**9,5 Go**
Hôte Fedora (OS + VS Code + Wireshark)	≈ 3 Go	≈ 3 Go
Total général	$\approx 9,5$ **Go**	$\approx 12,5$ **Go**

Avec 16 Go de RAM physique sur l'hôte, il reste une marge de sécurité de **3,5 à 6,5 Go**. Cette marge permet :

- d'absorber des pics ponctuels (compilation, snapshots VirtualBox),
- d'utiliser l'interface graphique de Kali Linux sans ralentissements,
- d'éventuellement augmenter légèrement les limites Docker si l'indexer Wazuh nécessite plus de ressources (tout en restant sous 4 Go).

→ **Décision :** **Surveillance proactive du swap :** Sur la VM SOC (Ubuntu), il est conseillé de réduire la tendance à utiliser le swap (*swappiness*) en ajoutant `vm.swappiness=10` dans `/etc/sysctl.conf`. Cela évitera que le système ne commence à swapper alors qu'il reste de la RAM libre.

8 Automatisation du laboratoire (Infrastructure as Code)

Pour garantir la reproductibilité du laboratoire et documenter sa construction, l'ensemble des machines virtuelles est défini via **Vagrant**, un outil de gestion d'infrastructure déclarative. Un fichier `Vagrantfile` décrit chaque VM (système d'exploitation, RAM, réseau) et des scripts de *provisioning* shell automatisent l'installation des dépendances (Docker, rsyslog, Python). Cette approche permet de reconstruire intégralement l'environnement en une seule commande (`vagrant up`) et constitue une bonne pratique d'ingénierie logicielle appliquée à la cybersécurité.

→ **Décision :** Le firewall OPNsense ne dispose pas d'une image Vagrant officielle. Il sera donc installé manuellement une première fois, tandis que les autres VMs (SOC, Cible, Kali) seront entièrement automatisées via Vagrant. Un script de *snapshot* VirtualBox pourra être ajouté pour sauvegarder l'état initial du firewall.

9 Recommandations techniques complémentaires

Cette section synthétise les points d'attention importants pour les prochaines semaines.

9.1 Snapshots Libvirt

→ **Décision :** Avant de démarrer la semaine 3 (collecte des logs) et surtout avant le déploiement de Wazuh en semaine 8, un **snapshot** de la VM SOC dans VirtualBox sera pris afin que en cas d'erreur de configuration ou de corruption des données, je puisse revenir instantanément à un état fonctionnel sans devoir tout réinstaller.

9.2 Synchronisation temporelle

→ **Décision :** Pour que les mesures de latence (délai entre événement et alerte) soient fiables, il est impératif que toutes les VMs soient synchronisées. Je configurerai `chrony` ou `systemd-timesyncd` pour pointer vers une source NTP commune (par exemple `pool.ntp.org`) ou, à défaut, ajuster manuellement les horloges avant chaque session de test avec `date -s`.

9.3 Journalisation des décisions

→ **Décision :** Le fichier `docs/decisions.md` mentionné dans la structure du dépôt Git doit être alimenté dès la semaine 0 avec les justifications des écarts décrits à la section 3. Cela facilitera la rédaction du rapport final et montrera une démarche d'ingénierie rigoureuse.

10 Conclusion de la semaine 0

L'architecture retenue pour le laboratoire virtualisé concilie les contraintes matérielles strictes (16 Go de RAM total, coût nul) avec les objectifs du projet : développer un

moteur de corrélation Python, le tester sur deux scénarios représentatifs, et le comparer objectivement à Wazuh. Les écarts par rapport au plan initial sont mineurs et justifiés par un gain en réalisme ou en fiabilité.

La topologie réseau, les flux de logs et la gestion des ressources ont été documentés de manière exhaustive. Le laboratoire est maintenant prêt pour la **semaine 1** : mise en place de l'infrastructure de base (firewall, routage, connectivité).

Prochaine étape : Déploiement du pare-feu OPNsense, configuration des règles de routage et tests de connectivité entre toutes les machines.